COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/13/2026	**DATE SUBMITTED:**	10/15/2026
NAMES:	Dimaguiba, Lorenzo James M. Urbano, Vince Andrew F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

ILO A:

Strategy	Algorithm	Analysis
Recursion	**Binary Search**	It is recursive in its method, after finding the middle element and comparing it with the search element, if there were no matches, depending if it was greater than or less than the search element, the function would call itself, setting the low or high position of the element in the position below or above the middle value as its parameter. Effectively halving the list to search. This continues till the element was found or the entire list was traversed.
Brute Force	**Bubble Sort**	Bubble sort brute forces by checking and comparing each and every element from start to finish, swapping with the right conditions till the function loop ends. It runs every check and solution each pass till all is sorted.
Backtracking	**Depth First Search**	The depth first search uses backtracking as it uses the concept of go-deep-then-retreat. If the value being searched is not found on the first try, it will check its steps again.
Greedy	**Selection Sort**	The selection sort is using a greedy algorithm because it automatically picks the best choice that can lead

		directly to the goal without considering the other values.
Divide and Conquer	**Merge Sort**	The merge sort uses Divide and Conquer because it divides the array into two until it can no longer be divided, then checks and combines the values.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	HOA12.1 > main.cpp > ...
	1 #include <iostream>
	2 #include <algorithm>
	3 const int MAX = 1000;
	4 int memo[MAX + 1];
	5 int getMinSteps (int n){
	6 if(n == 1) return 0;
	7 if (memo[n] != -1) return memo[n]; //solved everything
	8
	9 int r = 1 + getMinSteps(n - 1); // '-1' step . 'r' will contain the optimal answer
	10
	11 if (n % 2 == 0) r = std::min(r, 1 + getMinSteps(n / 2)); // "/2" step
	12
	13 if (n % 3 == 0) r = std::min(r, 1 + getMinSteps(n / 3)); // "/3" step
	14
	15 memo[n] = r; //save result. Forgetting the step, makes it the same as plain recursion.
	16 return r;
	17 }
	18
	19 int main(){
	20 int n = 10;
	21
	22 for (int i = 0; i < MAX; i++){
	23 memo[i] = -1;
	24 }
	25
	26 std::cout<<getMinSteps(n);
	27
	28 return 0;
	29 }
	30
	31

Analysis	First, a memo array is created which tracks which problems are solved. To do that, they are initialized as '-1' to mark them as "not solved" yet. The main goal is to find the most optimal way or get the least amount of steps to one. Once assigned, the function is called, it chooses a step from the 3 different steps from what's given. It then recurs the same steps, choosing the most optimal route. Once complete, it saves that result and returns it. Resolving the rest of the recurred functions to the original function, returning the final result.

Table 12-2. Memoization Implementation

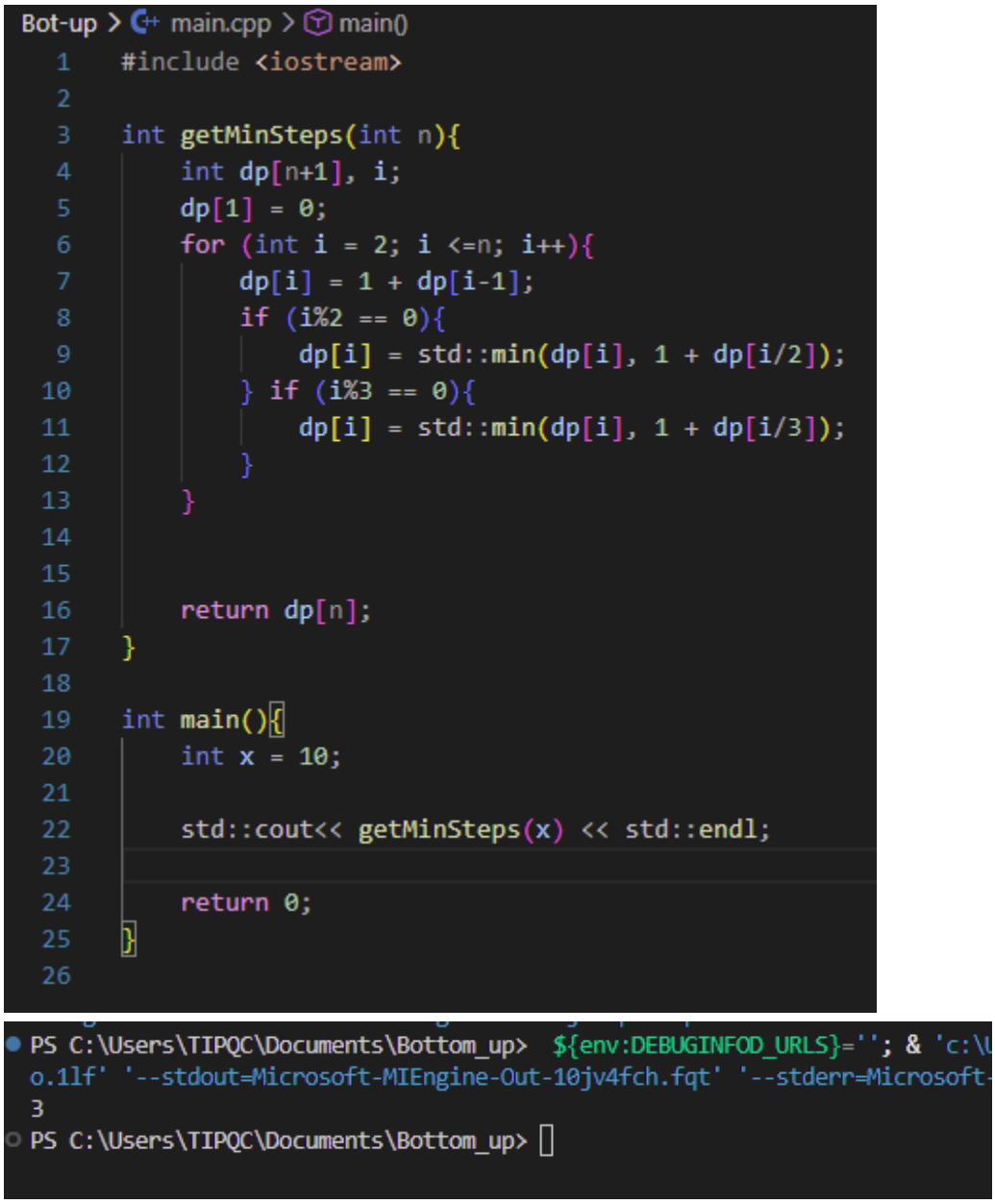
Screenshot	
Analysis	The function of the program is to get the minimum steps to one, but instead of starting from the largest number which is 10, it starts from the start which is one. It searches for the shortest path to one.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code:

main.cpp

```cpp
#include <iostream>
#include <vector>
#include <chrono>

// m = number of rows & n = number of columns
int paths(std::vector<std::vector<int>> const &matrix, int cost);
int find(std::vector<std::vector<int>> const &matrix, int m, int n, int cost);


int main(){
    int cost;
    std::vector<std::vector<int>> matrix = {
        {4,7,1,6},
        {6,7,3,9},
        {3,8,1,2},
        {7,1,7,3}
    };
    std::cout<<"Path Finder\n\nEnter cost: ";
    std::cin>>cost;

    //start timer
    auto start :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    //execution of func
    int total_paths = paths(matrix, cost);
    //end timer
    auto end :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    auto duration_us = std::chrono::duration_cast<std::chrono::microseconds>(d:end - start);

```

```
    //will find the number of paths equal to that cost
    std::cout<<"Number of paths to get cost "<< cost<< " is: "<< paths(matrix,cost) << std::endl;
    std::cout << "Execution time: " << duration_us.count() << " microseconds" << std::endl;


    return 0;
}


int paths(std::vector<std::vector<int>> const &matrix, int cost){
    //starting case
    if(matrix.size() == 0){
        return 0;
    }

    // the matrix
    int m = matrix.size();
    int n = matrix[0].size();
    return find(matrix, m - 1, n - 1, cost);
}

int find(std::vector<std::vector<int>> const &matrix, int m, int n, int cost){

    // starting case
    if (cost < 0){
        return 0;
    }

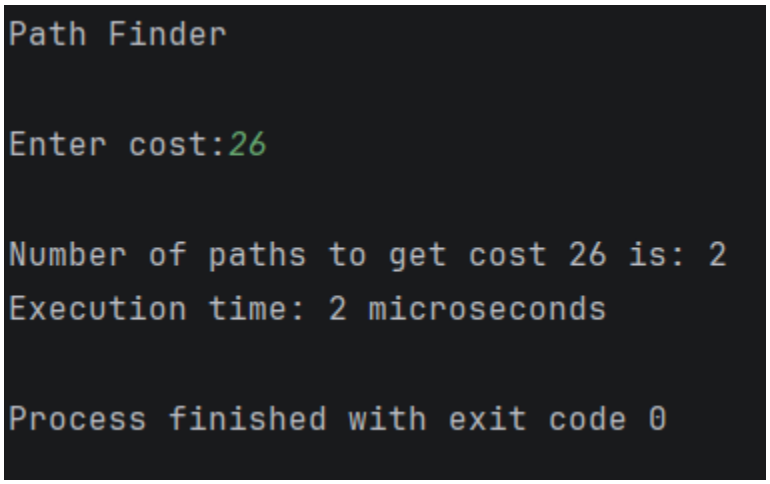
    //checks if it is at the first cell
    if (m == 0 && n == 0){
        if (cost - matrix[0][0] == 0){
            return 1;
        }
        else {
            return 0;
        }
    }

    //finds the pathway backwards (technically still going right and down if starting from the left)
    // goes left if it is the first row
    if (m == 0) {
        return find(matrix, m: 0, n - 1, cost - matrix[m][n]);
    }

    //goes up if it is the first row
    if (n == 0) {
        return find(matrix, m - 1, n: 0, cost - matrix[m][n]);
    }

    //recursion for counting paths
    return find(matrix, m - 1, n, cost - matrix[m][n]) + find(matrix, m, n -1 , cost - matrix[m][n]);

}
```

Output:	
Analysis:	In this code, we have created a 4 by 4 grid with int values. The grid will be used to help identify the number of unique paths that can be taken to find the value or cost based on the user's input. The function path is responsible for checking if the matrix is empty and if it is it will automatically return zero. The "find" function on the other hand uses top-down recursion to count the valid paths by checking backwards from the destination cell to the starting cell. After each step, it subtracts the current cell's value from the remaining target cost. If the remaining cost drops below zero, the branch will return zero. If the program successfully reaches the target path, it returns 1. Which resolves the recurred functions, ultimately returning and printing the number of paths. "Chrono" was utilized as well to time the execution of the functions, often taking 2-3 microseconds.

Conclusions/Learnings:

Dimaguiba:
Algorithms play a key role in the way we can solve problems. Whether to arrange or sort information, or solve complex problems, there is always an algorithm fit for the job. Even expensive algorithms like "brute-forcing" is still a viable strategy, provided they are used for specific purposes. Outside of the common algorithms we already know, like recursion (functions calling themselves) to Divide-and-Conquer algorithms, some new players have entered the block. Strategies like Dynamic Programming that takes note or saves results, avoiding having to solve the same problem over-and-over (Memoization as an example). The other is "Greedy" , that selects the functions it sees fit, attempting to get the most optimal solution (though not perfect). However, it does not mean they cannot be used together. The algorithms and combinations used, their effectiveness, may vary.

Urbano:
In this activity, I have learned more about different algorithm strategies such as recursion, brute force, backtracking, greedy algorithm, divide and conquer and dynamic programming. Recursion is basically about calling the same function again inside the function until it satisfies the given conditions from the function. Brute force algorithm uses a direct problem solving method to find the best solution for the correct solution. Backtracking is best used for tree graphs because it can be utilized for checking the root node to the child node and checking the roots again. A greedy algorithm is a type of algorithm that is used for solving optimization problems. Divide and Conquer is heavily used for different sorting algorithms like selection sort and merge sort. Lastly, dynamic programming is a technique used to solve a particular class of problems.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.